

AI Skin Screening Assistant

Project Type: AI-powered skin disease screening and consultation system

Technologies: Python, FastAPI, TensorFlow, FAISS, Google GenAI, React, Vite, Axios

Project Goal: Upload a skin image, get a preliminary risk-based screening result, and ask healthcare-related follow-up questions through a chatbot.

1. Project Overview

This project is an AI-assisted skin screening application designed to help users understand a visible skin concern before consulting a dermatologist. Users can upload a clear image of the affected skin region, and the system performs a preliminary assessment using a trained deep learning model. The app returns a predicted condition, confidence score, and risk level. It also includes a chatbot that answers general questions using a retrieval-augmented generation (RAG) pipeline backed by dermatology knowledge chunks.

The system does not replace diagnosis by a medical professional. It serves as a guided screening and educational tool.

2. Objectives

- Build a skin disease screening web application with AI-powered image analysis.
- Detect whether the uploaded image is a skin image or a non-skin object.
- Classify the skin condition from a predefined list of common conditions.
- Provide risk categorization: Low, Medium, or High.
- Give users an interactive chatbot for general guidance using trusted medical knowledge sources.
- Keep the system user-friendly and accessible through a modern frontend experience.

3. Features Implemented

Frontend

- Home page with landing screen and project overview.

Backend

- FastAPI server for image prediction and chat requests.

- Upload photo flow for possible skin examination.
- Risk results page with confidence progress bar.
- About page explaining the AI assistant and screening process.
- Floating chatbot UI for follow-up interaction.

- Image validation and non-skin detection before classification.
- TensorFlow model loading and prediction pipeline.
- Gemini-based chatbot with RAG response generation.

4. System Architecture

The application is split into two major parts: a Python backend and a React frontend.

Backend Components

- **skin_backend/main.py**: API entry point and model inference logic.
- **skin_backend/chatbot/rag.py**: RAG retrieval and Gemini integration.
- **skin_backend/chatbot/build_vector_db.py**: Builds FAISS vector database from knowledge files.
- **skin_backend/chatbot/embeddings.py**: Text embedding generation logic.
- **skin_backend/chatbot/safety.py**: Safety checks and blocked-content filtering.

Frontend Components

- **skin_frontend/src/App.jsx**: Routing for home, screen, result, and about pages.
- **skin_frontend/src/pages/Screen.jsx**: Photo upload and analysis flow.
- **skin_frontend/src/pages/Result.jsx**: Displays risk and prediction result.
- **skin_frontend/src/pages/chatbot.jsx**: Chat widget for user message handling.
- **skin_frontend/src/services/api.js**: API calls to the backend.

5. AI and Model Pipeline

The backend loads a trained EfficientNetB3-based model with a custom output head for skin disease classification. It also loads a MobileNetV2 model for non-skin object detection.

- Uploaded image is validated and converted to RGB.
- The image is checked for non-skin object signatures.
- Valid skin images are resized and passed to the prediction model.
- The model predicts a class from the known skin conditions.
- The application calculates prediction confidence and risk level.

The prediction returns a disease label, confidence percentage, and disclaimer text so that the result is framed as screening guidance rather than a diagnosis.

6. Risk Level Logic

The project uses a rule-based risk mapping layer to assign Low, Medium, or High risk based on the predicted disease class.

- **High Risk:** melanoma/cancer-related and other malignant skin lesions.
- **Medium Risk:** lupus, connective tissue disorders, bullous disease, vasculitis, systemic conditions.
- **Low Risk:** other conditions that are less urgent based on the configured category list.

7. Chatbot and RAG System

The chatbot is built around a retrieval-augmented generation design using a text knowledge base and FAISS vector search.

- Knowledge files are stored in the knowledge base folder.
- Text is chunked into pieces and converted into embeddings.
- FAISS indexes these embeddings for similarity search.
- When a user asks a question, the stored knowledge relevant to that message is retrieved.
- The app sends the retrieved content and user question to the Gemini model.
- Safety checks are incorporated before returning a response.

This provides general educational and follow-up guidance rather than a medical diagnosis.

8. Project Structure

- **skin_backend/** — trained model, API, chatbot logic, and vector DB data.
- **skin_backend/main.py** — FastAPI app and image prediction endpoints.
- **skin_backend/chatbot/** — RAG, embeddings, vector database creation, and safety logic.
- **skin_backend/chatbot/knowledge_base/** — educational content for many skin conditions.
- **skin_backend/chatbot/vector_db/** — generated FAISS index and documents metadata.
- **skin_frontend/** — React application with user interface pages and assets.
- **skin_frontend/src/pages/** — screen, result, home, and about flows.
- **skin_frontend/src/services/api.js** — frontend API client connection.

9. Key Implementation Details

- The backend exposes a root endpoint and prediction endpoint at `/predict`.

- The chat endpoint accepts a message and optional last detected condition.
- Frontend uses React Router to navigate between pages.
- A floating chatbot uses sessionStorage to remember the last predicted condition.
- Risk guidance and disclaimers are displayed to clarify that the result is not a formal diagnosis.

10. Technologies Used

Python

FastAPI

TensorFlow

Keras

NumPy

Pillow

FAISS

Google GenAI

React

Vite

Axios

11. Application Flow

1. User opens the frontend application.
2. User uploads a skin image on the screening page.
3. Frontend sends the image file to the backend /predict endpoint.
4. Backend validates the image and rejects non-skin objects.
5. Model predicts the most likely condition and confidence score.
6. Backend returns the disease name, confidence, and risk classification.
7. Frontend displays the screening result and recommendation information.
8. User can ask the chatbot questions for general guidance using the most recent condition context.

12. Project Outcome

The project successfully combines AI-based image classification with an accessible web interface and an intelligent dermatology-support chatbot. It demonstrates a usable prototype for preliminary skin condition screening and general information retrieval, while reinforcing that medical diagnosis should always involve professional evaluation.

13. Important Note

This project is a screening and educational application, not a substitute for medical diagnosis. It should be used as a preliminary assistive tool only, and any serious concern should be reviewed by a qualified healthcare professional.

Document generated for the AI Skin Screening Assistant project.